Prueba DevOps

En esta prueba se quiere evaluar tus conocimientos y habilidades prácticas en herramientas y metodologías clave del rol DevOps Senior: Git, GitHub Actions, Kubernetes, OpenShift, Helm y ArgoCD.

La prueba busca validar la capacidad para:

- Comprender y aplicar conceptos de integración continua y despliegue continuo (CI/CD).
- Diseñar e implementar pipelines que automaticen la entrega de aplicaciones.
- Trabajar bajo el enfoque GitOps, manteniendo a Git como la fuente de la verdad.
- Identificar, analizar y resolver problemas en entornos de Kubernetes y OpenShift.

1. Git

Pregunta:

Explica la diferencia entre git rebase y git merge. ¿En qué situaciones preferirías cada uno y por qué?

Respuesta:

Las dos sirven para llevar cambios de una rama a otra, pero lo hacen distinto. Merge junta las ramas con un commit de fusión y el historial queda tal cual pasó, con sus bifurcaciones. Rebase toma los commits de mi rama y los vuelve a aplicar encima de la otra, así que se crean commits nuevos y el historial queda en línea recta.

Yo uso rebase en mi rama de trabajo cuando quiero ponerla al día con main o limpiar mis commits antes de abrir el Pull Request. Uso merge cuando voy a integrar a una rama compartida como main, porque no reescribe nada y se puede ver quién cambió qué y cuándo. La regla que sigo es no hacer rebase de ramas que ya subí y que otras personas están usando, porque les cambia el historial y les genera conflictos.

2. GitHub Actions

Pregunta:

En un workflow de GitHub Actions, ¿cuál es la diferencia entre jobs.<job_id>.runs-on y jobs.<job_id>.container? Da un ejemplo de uso para cada caso.

Respuesta:

runs-on dice en qué máquina corre el job, o sea el runner. Por ejemplo, runs-on: ubuntu-latest usa un runner que pone GitHub, y si necesito llegar a una red interna uso uno propio con labels, algo como runs-on: [self-hosted, linux].

container, en cambio, hace que los pasos del job corran dentro de una imagen Docker, pero siempre sobre el runner que se definió en runs-on. Sirve para tener un entorno fijo con las herramientas ya instaladas. Por ejemplo, runs-on: ubuntu-latest con container: node:20-alpine, para compilar con esa versión exacta de Node sin depender de lo que traiga el runner.

Mi experiencia con pipelines viene sobre todo de Azure DevOps, donde el concepto es muy parecido (pool de agentes y contenedores para los jobs).

3. Kubernetes

Pregunta:

¿Cuál es la diferencia entre un Deployment, un StatefulSet y un DaemonSet en Kubernetes? Explica un caso real donde usarías cada uno.

Respuesta:

El Deployment es para aplicaciones sin estado, donde los pods son todos iguales y se pueden reemplazar sin problema. Es lo que uso para una API o un frontend, porque permite escalar y hacer actualizaciones graduales.

El StatefulSet es para cuando cada réplica necesita su propia identidad y su propio disco, con un nombre fijo (pod-0, pod-1) y un orden de arranque. Lo usaría para una base de datos como PostgreSQL o para algo como Kafka.

El DaemonSet pone un pod en cada nodo del clúster. Lo usaría para un agente de logs como Fluent Bit o uno de monitoreo, que tiene que estar en todos los nodos.

4. Kubernetes

Pregunta:

Si un Pod entra en estado CrashLoopBackOff, ¿qué pasos seguirías para diagnosticar y resolver el problema?

Respuesta:

Primero miro qué dice el pod con kubectl describe pod, sobre todo los Events, el último estado y el código de salida. Eso ya da pistas: un 137 suele ser falta de memoria (OOMKilled) y un 1 suele ser un error de la propia aplicación.

Después reviso los logs de la ejecución anterior con kubectl logs --previous, porque el contenedor actual puede haber muerto antes de escribir algo. Si hay varios contenedores en el pod, indico cuál con -c. Esto se parece a lo que hago en AWS con CloudWatch cuando reviso los logs de una implementación para ver por qué falló.

Las causas que más se repiten son una variable de entorno, ConfigMap o Secret que falta, un comando mal escrito, una liveness probe muy estricta que mata el pod, un límite de memoria muy bajo o una dependencia que no responde, como la base de datos. Si ni así encuentro nada, entro al contenedor con kubectl exec o lanzo uno de depuración para probar a mano.

Cuando encuentro la causa, la corrijo en el manifiesto, redespliego y verifico con kubectl rollout status. Si el flujo es GitOps, el arreglo lo hago en Git y no directamente en el clúster.

5. OpenShift

Pregunta:

¿Qué ventajas ofrece OpenShift sobre un clúster de Kubernetes estándar en cuanto a seguridad y gestión de aplicaciones?

Respuesta:

No he trabajado directamente con OpenShift, pero entiendo que es una distribución de Kubernetes con muchas cosas ya integradas y más controles de seguridad por defecto.

En seguridad, lo principal son las Security Context Constraints, que limitan lo que puede hacer un pod. Por defecto los contenedores no corren como root y reciben un usuario aleatorio, algo que en Kubernetes estándar hay que configurar a mano. También trae autenticación integrada con OAuth, RBAC y un registro interno de imágenes.

En gestión, trae una consola web completa, Routes para exponer las aplicaciones, builds integrados (S2I), Operators desde OperatorHub, monitoreo y logging incluidos, y soporte empresarial de Red Hat. Es decir, se gasta menos tiempo armando el ecosistema alrededor del clúster.

6. Helm

Pregunta:

Explica qué son los values.yaml en Helm y cómo permiten la parametrización de despliegues. ¿Qué diferencia hay entre values.yaml y values.override.yaml?

Respuesta:

Helm empaqueta los manifiestos de Kubernetes en un chart, y los templates del chart no llevan los datos fijos sino variables, como la imagen, las réplicas o los recursos. El archivo values.yaml es donde están los valores por defecto de esas variables. Así el mismo chart sirve para varios ambientes cambiando solo los valores.

Esos valores se pueden reemplazar al instalar con -f otro-archivo.yaml o con --set, y lo último que se aplica gana. Sobre values.override.yaml, entiendo que no es un nombre especial de Helm, sino una

convención: un archivo aparte con los valores de un ambiente concreto (por ejemplo producción) que se pasa con -f y pisa lo que dice values.yaml.

7. ArgoCD

Pregunta:

¿Qué significa que ArgoCD implemente el concepto de “Git como fuente de la verdad”? ¿Qué ventajas trae frente a un pipeline CI/CD tradicional?

Respuesta:

Significa que lo que debe estar desplegado se define en Git, y ArgoCD se encarga de comparar todo el tiempo lo que hay en el clúster con lo que dice Git, y de igualarlo. Nadie tiene que "empujar" el cambio al clúster: ArgoCD lo trae desde el repositorio.

Frente a un pipeline tradicional, que despliega por su cuenta, veo varias ventajas. El pipeline no necesita credenciales del clúster, lo que es más seguro. Cada cambio queda auditado en el historial de Git y volver atrás es tan simple como un git revert. Se detectan y corrigen las diferencias entre el clúster y Git (drift). Y si se pierde el clúster, se puede recrear desde el repositorio.

8. ArgoCD + Helm

Pregunta:

¿Cómo integras un chart de Helm dentro de una aplicación de ArgoCD? Explica los pasos y parámetros clave que deben configurarse en el manifiesto Application.

Respuesta:

Primero tengo el chart dentro de un repositorio, por ejemplo en charts/demo-app. Luego creo un manifiesto Application y lo aplico una sola vez con kubectl apply. A partir de ahí ArgoCD renderiza el chart con Helm y lo sincroniza al clúster.

Los parámetros más importantes dentro de spec son: source.repoURL (el repositorio), source.targetRevision (la rama, tag o commit), source.path (la carpeta del chart) y source.helm.valueFiles si quiero indicar qué archivos de valores usar. También destination.server y destination.namespace, que dicen a dónde se despliega, y project. Por último, syncPolicy con automated, prune y selfHeal, para que sincronice solo, borre lo que se elimine de Git y corrija cambios manuales. CreateNamespace=true sirve para que cree el namespace si no existe.

9. Integración GitOps

Pregunta:

En un flujo GitOps, ¿qué pasaría si alguien modifica directamente un recurso en el clúster de Kubernetes sin pasar por el repositorio Git? ¿Cómo se comportaría ArgoCD?

Respuesta:

ArgoCD notaría que lo que está en el clúster ya no coincide con Git y marcaría la aplicación como OutOfSync. Qué hace después depende de la configuración: si tiene selfHeal activado, revierte el cambio automáticamente y deja el recurso como dice Git. Si no lo tiene, solo avisa y hay que sincronizar a mano.

Eso es justo lo que se busca con GitOps: que la única forma de cambiar algo sea a través de Git, y que los cambios hechos a mano no se queden sin que nadie lo sepa.

Ejercicio Práctico

Enunciado:

Crea un pipeline básico en GitHub Actions (.github/workflows/deploy.yaml) que haga lo siguiente:

1. Se ejecute cuando haya un push a la rama main.
2. Compile y construya una imagen Docker de una aplicación simple (por ejemplo, Node.js o Python).
3. Etiquete la imagen con el commit SHA y la suba a un container registry (DockerHub o Quay.io).
4. Aplique un Helm chart o manifiesto YAML a un clúster de Kubernetes / OpenShift para desplegar la aplicación.
5. El despliegue debe seguir un enfoque GitOps: el pipeline debe actualizar un archivo values.yaml o un manifiesto de despliegue en el repositorio Git y permitir que ArgoCD sincronice los cambios hacia el clúster.

El entregable debe incluir:

- El workflow de GitHub Actions en YAML.
- Un ejemplo de values.yaml con la referencia de la imagen.
- Un manifiesto Application de ArgoCD que gestione la aplicación.

Posteriormente realizar la sustentación del ejercicio.

Cómo funciona mi solución

Cuando hay un push a main, el workflow tiene dos jobs. El primero construye la imagen Docker y la sube a Docker Hub con el commit SHA como tag. El segundo actualiza el campo image.tag en charts/demo-app/values.yaml y hace commit al repositorio. Después, ArgoCD detecta ese commit y sincroniza el chart con el clúster.

Por eso el pipeline no se conecta al clúster ni ejecuta helm upgrade: el despliegue lo hace ArgoCD, que es lo que pide el enfoque GitOps. Para evitar que el commit del pipeline dispare otra ejecución en bucle, el workflow ignora los cambios en charts/** y el commit lleva [skip ci]. Usé el SHA como tag porque no cambia y deja claro qué versión está desplegada, a diferencia de latest. Las credenciales de Docker Hub van en GitHub Secrets (DOCKERHUB_USERNAME y DOCKERHUB_TOKEN).

Entregable 1: .github/workflows/deploy.yaml

```
# Ubicación en el repo: .github/workflows/deploy.yaml
name: deploy

on:
  push:
    branches: [main]
    # Evita bucle infinito: el propio pipeline hace commit en charts/
    paths-ignore:
      - 'charts/**'
      - 'argocd/**'
      - '**.md'
```

```yaml
# Un solo despliegue a la vez (evita conflictos al hacer push del values.yaml)
concurrency:
  group: deploy-main
  cancel-in-progress: false

permissions:
  contents: write   # necesario para commitear el values.yaml

env:
  IMAGE_NAME: demo-app

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    outputs:
      tag: ${{ steps.vars.outputs.tag }}
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Definir tag (commit SHA)
        id: vars
        run: echo "tag=${GITHUB_SHA}" >> "$GITHUB_OUTPUT"

      - name: Setup Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login a Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Build y push de la imagen
        uses: docker/build-push-action@v6
        with:
          context: ./app
          push: true
          tags: |
            ${{ secrets.DOCKERHUB_USERNAME }}/${{ env.IMAGE_NAME }}:${{ steps.vars.outputs.tag }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

  update-gitops:
    needs: build-and-push
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4
        with:
          ref: main

      - name: Actualizar image.tag en values.yaml
        run: |
          yq -i '.image.tag = "${{ needs.build-and-push.outputs.tag }}"' charts/demo-app/values.yaml
```

```
      - name: Commit y push (ArgoCD sincroniza desde aquí)
        run: |
          git config user.name  "github-actions[bot]"
          git config user.email "github-actions[bot]@users.noreply.github.com"
          git add charts/demo-app/values.yaml
          git commit -m "chore(gitops): imagen ${{ needs.build-and-push.outputs.tag }} [skip ci]"
          git pull --rebase origin main
          git push origin main
```

Entregable 2: charts/demo-app/values.yaml

```
# Ubicación en el repo: charts/demo-app/values.yaml
# El pipeline actualiza SOLO image.tag con el commit SHA (yq). ArgoCD detecta el cambio y
sincroniza.
replicaCount: 2

image:
  repository: docker.io/straikerg/demo-app
  tag: "0000000000000000000000000000000000000000"   # lo reemplaza el pipeline
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 80
  targetPort: 3000

resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 256Mi

# Solo OpenShift: expone la app con una Route
route:
  enabled: false
  host: ""
```

Entregable 3: argocd/application.yaml

```
# Ubicación en el repo: argocd/application.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: demo-app
  namespace: argocd            # en OpenShift GitOps: openshift-gitops
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default
  source:
    repoURL: https://github.com/StraikerG/demo-devops.git
    targetRevision: main
    path: charts/demo-app      # carpeta del chart Helm
    helm:
      valueFiles:
        - values.yaml
```

```
  destination:
    server: https://kubernetes.default.svc
    namespace: demo-app
  syncPolicy:
    automated:
      prune: true              # borra del clúster lo que se elimine de Git
      selfHeal: true           # revierte cambios manuales en el clúster
    syncOptions:
      - CreateNamespace=true
    retry:
      limit: 3
      backoff:
        duration: 10s
        factor: 2
        maxDuration: 3m
```